

Bitácora: FFT

Contreras Reyes Orlando

July 31, 2026

Abstract

Este documento recoge, de forma clara y ordenada, las definiciones, explicaciones y ejemplos relativos al concepto de *orden multiplicativo* y *raíces primitivas* módulo n . Incluye demostraciones cortas, el criterio práctico para probar primitividad y ejemplos desarrollados paso a paso (incluyendo el algoritmo de Euclides cuando procede).

1 Introducción

Para hablar de la transformada de Fourier, es necesario entender primero ¿Qué es una transformada? ¿Cuál es?, este documento pretende explicar los conceptos necesarios para su entendimiento, así como su aplicación en la computación de la transformada rápida de Fourier (FFT).

Contents

1	Introducción	1
1.1	Transformada de Fourier	3
1.2	Propiedades de la Transformada de Fourier	4
1.2.1	Tabla de Fourier	6
1.3	Notacion $\{\}$	7
1.4	DFT Discrete Fourier Transform	7
1.4.1	Ejercicio practico	8
1.5	DFT Aplicada	9
1.5.1	Linea Continua	10
1.5.2	Propiedades de la DFT	10
1.5.3	Senoidal con 4 puntos	12
1.5.4	Pulso Cuadrado	14
2	Algoritmos	16
2.1	¿Qué es un algoritmo?	16
2.2	¿Cuál es el algoritmo de sumar dos números?	16
2.3	¿Cuál es el algoritmo de multiplicar dos números?	17
2.4	¿Cuál es el algoritmo de sumar dos matrices?	18
2.5	¿Cuál es el algoritmo de multiplicar dos matrices?	19
3	Definición de Complejidad Computacional	20
3.1	Objetivo del Análisis de Complejidad	20
3.2	Definición Formal de Big-O	21
3.3	Ejemplo de Complejidad Lineal	21
3.4	Importancia	22
4	Complejidad de la DFT	23
4.1	Cantidad Real de Operaciones en la DFT	23
4.2	Interpretación	25
4.3	FFT	25
4.3.1	Algoritmo	25
4.4	implementacion en un microcontrolador	25
5	Anexos	26
I	Transformada de Fourier Completa	26
II	Linealidad de la transformada de Fourier	26
III	Convolución	26
IV	Construccion de la Matriz de Fourier	27

Una transformación integral es una operación lineal que convierte una función $\mathbf{f}(x)$ en otra función $\tilde{\mathbf{f}}(s)$ a través de una integral definida. La forma general de una transformación integral es:

$$\tilde{\mathbf{f}}(s) = \int_a^b K(s, x)\mathbf{f}(x)dx$$

donde $K(s, x)$ es el núcleo de la transformación, que determina cómo se relacionan las funciones de entrada y salida. Las transformaciones integrales son herramientas que permiten cambiar de un dominio a otro, facilitando el análisis y la solución de problemas en diversas áreas de las matemáticas y la física.

Por ejemplo, la transformada de Fourier es una transformación integral que convierte una función del tiempo en una función de frecuencia, lo que es fundamental para el análisis de señales y sistemas.

Algo importante a destacar es la linealidad que las transformaciones integrales poseen, lo que significa que si tenemos dos funciones $\mathbf{f}_1$ y $\mathbf{f}_2$, y dos constantes a y b [2, 3], entonces:

$$c \cdot \tilde{\mathbf{f}}\{s\} = a \cdot \tilde{\mathbf{f}}\{s\}_1 + b \cdot \tilde{\mathbf{f}}\{s\}_2 \quad (1)$$

Ejemplos de transformaciones integrales y su uso

- **Laplace:** Utilizada para ecuaciones diferenciales y sistemas en dominio temporal
- **Fourier:** Utilizada para análisis de señales convirtiendo del dominio temporal al dominio de frecuencia
- **Mellin:** Utilizada en análisis asintótico y teoría de números
- **Z:** Utilizada en procesamiento de señales discretas y análisis de sistemas discretos

1.1 Transformada de Fourier

La transformada de Fourier es un modelo matemático que descompone una función en sus componentes de frecuencia. Es una herramienta fundamental en el análisis de señales, procesamiento de imágenes, física y muchas otras áreas. Como previamente mencionamos permite convertir una función del dominio temporal (o espacial) a una función del dominio de la frecuencia, lo que facilita el análisis y la manipulación de señales, un ejemplo visual se muestra en la figura 1, en donde se puede apreciar como se trabaja en el dominio de la transformada de Fourier para después aplicar la transformada inversa y regresar al dominio inicial.

La transformada de Fourier se define como:

$$\tilde{\mathbf{f}}\{s\} = \int_{-\infty}^{\infty} \mathbf{f}(x)e^{-(2\pi i)x}dx \quad (2)$$

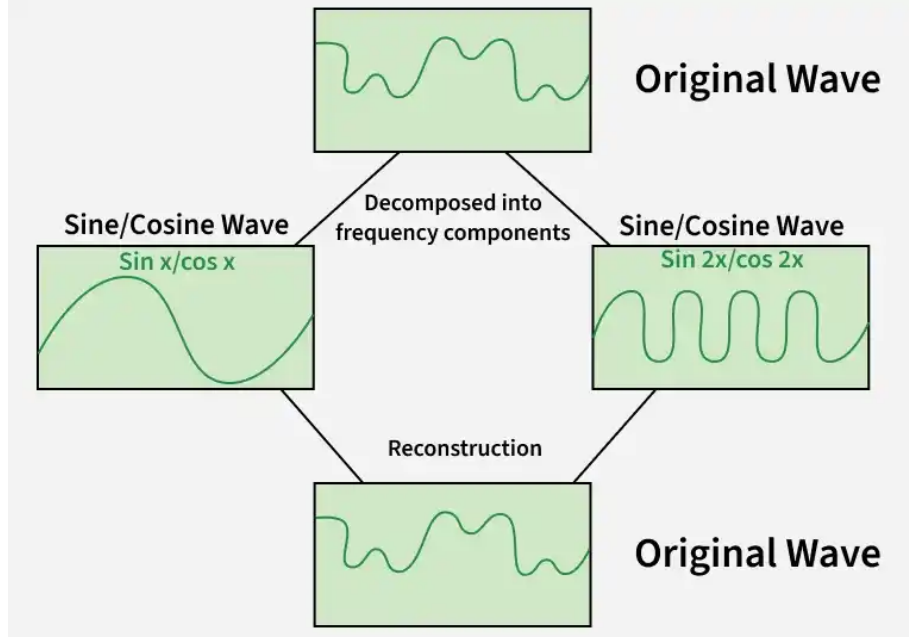


Figure 1: Transformada Fourier e inversa de Fourier

Es importante mencionar que la transformada de Fourier tiene una propiedad de inversibilidad, lo que significa que se puede recuperar la función original a partir de su transformada utilizando la fórmula inversa:

$$\mathbf{f}(x) = \int_{-\infty}^{\infty} \tilde{f}\{s\} e^{(2\pi i)kx} ds \quad (3)$$

También se puede representar la transformada de Fourier y su inversa mediante $\mathcal{F}(k)$ y $\mathcal{F}^{-1}(x)$.

Asimismo, muchos autores suelen utilizar w en lugar de $2\pi k$ para representar la frecuencia angular ¹, lo que da lugar a la siguiente formulación de la transformada de Fourier y su inversa: [5]

$$\tilde{f}\{s\} = \int_{-\infty}^{\infty} \mathbf{f}(t) e^{-iwt} dt \quad \text{y} \quad \mathbf{f}(x) = \int_{-\infty}^{\infty} \tilde{f}\{s\} e^{iwx} dw \quad (4)$$

Siendo esta la definición que se utilizará en el documento.

1.2 Propiedades de la Transformada de Fourier

Algunas propiedades de la transformada de Fourier serían las siguientes:

¹Es importante mencionar que esto es una versión con ciertos parámetros, esto rompe la simetría de la transformada; para ver la fórmula generalizada véase la Sección I.

- Linealidad: La transformada de Fourier es una operación lineal, lo que significa que la transformada de una suma de funciones es igual a la suma de las transformadas de cada función. Como se mostró previamente en la ecuación 1.²

$$\tilde{f}\{af(x) + bg(x)\} = a\tilde{f}\{f(x)\} + b\tilde{f}\{g(x)\}$$

- Desplazamiento: Si una función se desplaza en el dominio del tiempo, su transformada se multiplica por un factor exponencial complejo.

$$\tilde{f}\{f(x - x_0)\} = e^{-j\omega x_0} \tilde{f}\{f(x)\}$$

- Escalamiento: Si una función se escala en el dominio del tiempo, su transformada se escala inversamente en el dominio de la frecuencia.

$$\tilde{f}\{f(ax)\} = \frac{1}{|a|} \tilde{f}\left\{\frac{f(x)}{a}\right\}$$

- Conjugación: La transformada de Fourier de la conjugada compleja de una función es igual a la conjugada compleja de la transformada de Fourier de la función original.

$$\tilde{f}\{f^*(x)\} = \tilde{f}\{f(x)\}^*$$

- Convolución: La transformada de Fourier de la convolución de dos funciones es igual al producto de las transformadas de Fourier de cada función.³

$$\tilde{f}\{f(x) * g(x)\} = \tilde{f}\{f(x)\} \cdot \tilde{f}\{g(x)\}$$

²Como demostración se puede usar la definición de la transformada de Fourier y aplicar la propiedad de linealidad de la integral como se muestra en el Anexo II.

³Siendo * el operador de convolución, definido en la sección III

1.2.1 Tabla de Fourier

A continuación se muestra una tabla con algunas funciones comunes y sus respectivas transformadas de Fourier:

$$\begin{aligned}
 \tilde{f}\{1\} &= 2\pi\delta(\omega) \\
 \tilde{f}\{\delta(t)\} &= 1 \\
 \tilde{f}\{u(t)\} &= \pi\delta(\omega) + \frac{1}{i\omega} \\
 \tilde{f}\{e^{-at}u(t)\} &= \frac{1}{a + i\omega}, \quad a > 0 \\
 \tilde{f}\{e^{-a|t|}\} &= \frac{2a}{a^2 + \omega^2}, \quad a > 0 \\
 \tilde{f}\{\sin(\omega_0 t)\} &= \frac{\pi}{i}[\delta(\omega - \omega_0) - \delta(\omega + \omega_0)] \\
 \tilde{f}\{\cos(\omega_0 t)\} &= \pi[\delta(\omega - \omega_0) + \delta(\omega + \omega_0)] \\
 f\{\delta(\omega)\} &= \frac{1}{2\pi} \\
 f\{1\} &= \delta(t) \\
 f\left\{\pi\delta(\omega) + \frac{1}{i\omega}\right\} &= u(t) \\
 f\left\{\frac{1}{a + i\omega}\right\} &= e^{-at}u(t), \quad a > 0 \\
 f\left\{\frac{2a}{a^2 + \omega^2}\right\} &= e^{-a|t|}, \quad a > 0 \\
 f\left\{\frac{\pi}{i}[\delta(\omega - \omega_0) - \delta(\omega + \omega_0)]\right\} &= \sin(\omega_0 t) \\
 f\{\pi[\delta(\omega - \omega_0) + \delta(\omega + \omega_0)]\} &= \cos(\omega_0 t)
 \end{aligned}$$

$$\mathbf{1)} \quad f(t) = \delta(t) \quad \Longrightarrow \quad F(\omega) = 1$$

$$\mathbf{2)} \quad f(t) = e^{-2t}u(t) \quad \Longrightarrow \quad F(\omega) = \frac{1}{2 + j\omega}$$

$$\mathbf{3)} \quad f(t) = e^{-3|t|} \quad \Longrightarrow \quad F(\omega) = \frac{6}{9 + \omega^2}$$

$$\mathbf{4)} \quad f(t) = \cos(4t) \quad \Longrightarrow \quad F(\omega) = \pi[\delta(\omega - 4) + \delta(\omega + 4)]$$

$$\mathbf{5)} \quad f(t) = \sin(5t) \quad \Longrightarrow \quad F(\omega) = \frac{\pi}{j}[\delta(\omega - 5) - \delta(\omega + 5)]$$

$$6) f(t) = \text{rect}\left(\frac{t}{2}\right) \quad \Longrightarrow \quad F(\omega) = \int_{-1}^1 e^{-j\omega t} dt = \frac{2 \sin(\omega)}{\omega}$$

$$7) f(t) = e^{-t^2} \quad \Longrightarrow \quad F(\omega) = \sqrt{\pi} e^{-\omega^2/4}$$

$$8) f(t) = 1 \quad \Longrightarrow \quad F(\omega) = 2\pi \delta(\omega)$$

1.3 Notacion $\{\}$

Los Funtores son una herramienta matemática que se utiliza para describir la relación entre dos categorías, en este caso, entre el dominio del tiempo y el dominio de la frecuencia. En el contexto de la transformada de Fourier, los funtores se utilizan para representar la transformación de una función del dominio del tiempo a una función del dominio de la frecuencia. La notación $\{\}$ se utiliza para denotar un functor, y en este caso, se puede interpretar como la aplicación de la transformada de Fourier a una función $f(t)$, lo que da como resultado la función $F(\omega)$ en el dominio de la frecuencia. Es decir, $\{f(t)\}$ representa la función transformada de Fourier de $f(t)$, lo que se puede expresar como:

$$\{f(t)\} = F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt$$

Esta notación es útil para simplificar la representación de la transformada de Fourier y para enfatizar la relación entre las funciones en los dos dominios. Además, permite expresar de manera concisa la aplicación de la transformada de Fourier a diferentes funciones, lo que facilita el análisis y la manipulación de señales en el dominio de la frecuencia. Es importante destacar que el argumento de la notación $\{\}$ debe ser una función y no un número, ya que la transformada de Fourier se aplica a funciones y no a valores individuales.

1.4 DFT Discrete Fourier Transform

Debemos considerar que el mundo real es de manera continua pero a la hora de procesar una señal debe ser tomada por un microcontrolador, lo que implica que la señal debe ser muestreada, es decir, convertida a una señal discreta. Para esto se utiliza la DFT (Discrete Fourier Transform), que es una versión discreta de la transformada de Fourier, además es un caso especial de la transformada Z. [6]

La DFT es ampliamente utilizada en procesamiento de señales digitales, análisis de espectro, compresión de datos y muchas otras aplicaciones. Expresar la DFT en términos de sumatoria permite observarla desde otra perspectiva. También puede ser vista como un método para convertir una secuencia de números complejos en otra secuencia de números complejos, lo que es útil para analizar la frecuencia de una señal.

digital, de estos dos métodos se puede hacer un cambio de variable para observarla como una matriz de Fourier, lo que facilita su implementación computacional.

La DFT se define como: ⁴

$$X[k] = \sum_{j=0}^{N-1} x[j]e^{-(2\pi i)kj/N}, \quad k = 0, 1, \dots, N-1 \quad (5)$$

Haciendo un cambio de variable $\omega = e^{-(2\pi i)/N}$, la DFT se puede expresar como:

$$X[k] = \sum_{j=0}^{N-1} x[j]\omega^{jk}$$

La matriz de Fourier se construye a partir de los factores ω^{jk} , lo que da lugar a una matriz de $N \times N$ con elementos que son potencias de ω . La DFT se puede calcular multiplicando esta matriz por el vector de entrada $x[j]$, lo que da como resultado el vector de salida $X[k]$.

$$\begin{bmatrix} \hat{f}_0 \\ \hat{f}_1 \\ \hat{f}_2 \\ \vdots \\ \hat{f}_{N-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \dots & \omega^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{bmatrix} \quad (6)$$

Sin embargo, el cálculo directo de la DFT tiene una complejidad de $O(N^2)$, lo que puede ser ineficiente para grandes valores de N . Para abordar este problema, se desarrolló la Transformada Rápida de Fourier (FFT), que es un algoritmo eficiente para calcular la DFT con una complejidad de $O(N \log N)$ como se puede ver en la fig 2.

1.4.1 Ejercicio practico

Es importante hacer mencion que el factor

$$w = e^{-(2\pi i)/N}$$

es una raíz de la unidad, lo que significa que $w^N = 1$. Esto se debe a que w representa un punto en el plano complejo que se encuentra en la circunferencia unitaria, y al elevarlo a la potencia N , se completa una vuelta completa alrededor de la circunferencia, regresando al punto inicial. Esta propiedad es fundamental para el funcionamiento de la DFT y la FFT, ya que permite descomponer una señal en sus componentes de frecuencia utilizando estas raíces de la unidad.

⁴Existe una amplia variedad de notaciones, se utilizará esta por ser la más común y basandonos en la de la referencia [9], aunque también se puede encontrar con X_k o $F(k)$ en lugar de F_k .

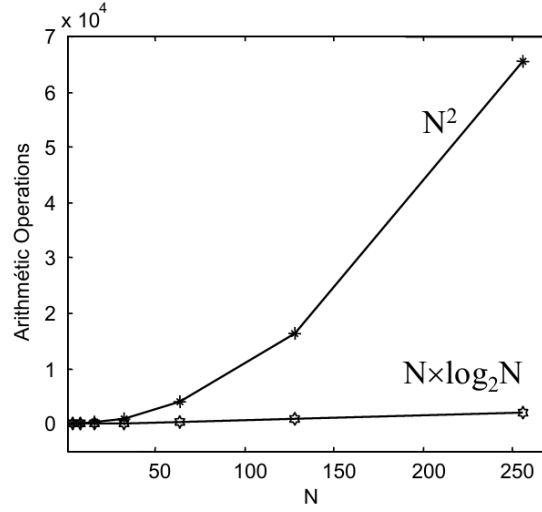


Figure 2: Comparación de la complejidad de la DFT y la FFT

1.5 DFT Aplicada

Definiendose

$$W_N = e^{-(2\pi j)/N}$$

Desarrollando la sumatoria con un $N = 4$ se obtiene la siguiente matriz:

$$\begin{aligned} X[0] &= x[0]W_4^{0 \cdot 0} + x[1]W_4^{0 \cdot 1} + x[2]W_4^{0 \cdot 2} + x[3]W_4^{0 \cdot 3} \\ X[1] &= x[0]W_4^{1 \cdot 0} + x[1]W_4^{1 \cdot 1} + x[2]W_4^{1 \cdot 2} + x[3]W_4^{1 \cdot 3} \\ X[2] &= x[0]W_4^{2 \cdot 0} + x[1]W_4^{2 \cdot 1} + x[2]W_4^{2 \cdot 2} + x[3]W_4^{2 \cdot 3} \\ X[3] &= x[0]W_4^{3 \cdot 0} + x[1]W_4^{3 \cdot 1} + x[2]W_4^{3 \cdot 2} + x[3]W_4^{3 \cdot 3} \end{aligned}$$

Pudiendo escribir esto en forma matricial como:

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix} \Rightarrow \vec{X} = \widehat{W_4^{kn}} \vec{x}$$

Calculando Factores :

$$W^0 = W^4 = +1$$

$$W^1 = -i$$

$$W^2 = -1$$

$$W^3 = +i$$

Valores adicionales:

$$W^6 = W^{4+2} = W^2 = -1$$

$$W^9 = W^{4*2+1} = W^1 = -i$$

Quedando en una matriz de la siguiente forma:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}$$

1.5.1 Línea Continua

Considerando la línea continua como el vector

$$\vec{x} = \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix}$$

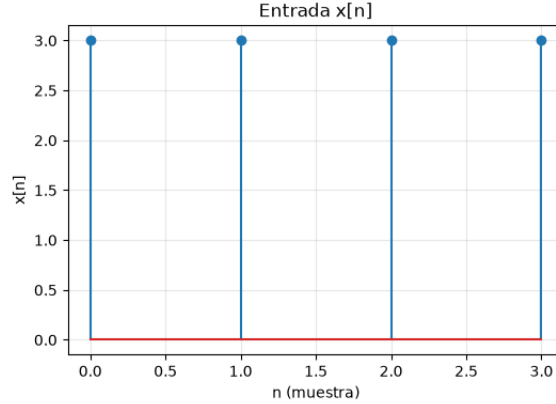


Figure 3: LineaContIN

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix} \begin{bmatrix} 3 \\ 3 \\ 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 + 3 + 3 + 3 \\ 3 - 3i - 3 + 3i \\ 3 - 3 + 3 - 3 \\ 3 + 3i - 3 - 3i \end{bmatrix} = \begin{bmatrix} 12 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Siendo la Transformada Discreta de la función $f(x) = 3$ la siguiente

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} 12 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

1.5.2 Propiedades de la DFT

Además esto nos sirve de evidencia para ver que las propiedades aplicadas en la presentación funcionaron

además de obtener una nueva propiedad siendo que

$$W_4^3 = -W_4^1$$

Demostrándola de la siguiente forma

$$W_N^{N-m} = \frac{W_N^N}{W_N^m} = \frac{1}{W_N^m} = \frac{1}{e^{-2\pi jm/N}} = (e^{-2\pi jm/N})^{-1} = e^{2\pi jm/N} = -W_N^m \text{ Siendo } m \in \mathbb{Z} \neq 0$$

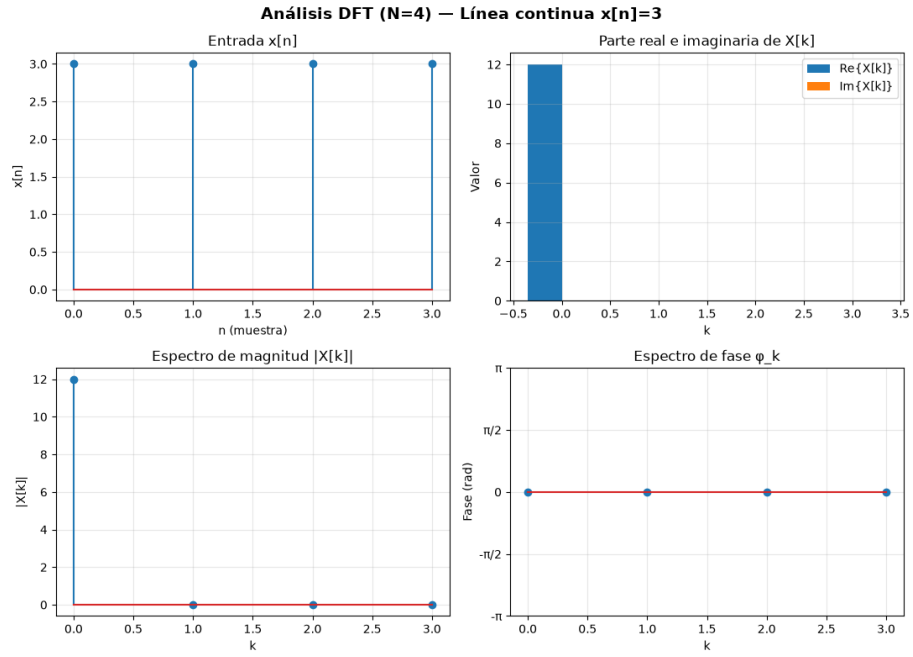


Figure 4: LineaOUT

Enfocandonos en la matriz de Fourier W_4^{kn}

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \xrightarrow[\text{propiedad 2}]{W_4^6=W_4^2, W_4^9=W_4^1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^2 \\ 1 & W_4^3 & W_4^2 & W_4^3 \end{bmatrix} \xrightarrow[\text{propiedad 3 y 1}]{W_4^2=-1, W_4^4=1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & W_4^3 \\ 1 & -1 & 1 & -1 \\ 1 & W_4^3 & -1 & W_4^1 \end{bmatrix}$$

Esto reduce la matriz a una computarizar una solo kernel (W)

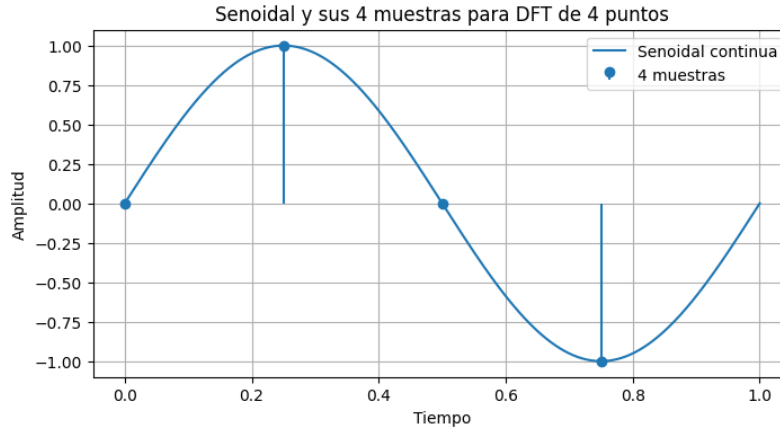
$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix}$$

1.5.3 Senoidal con 4 puntos

Para la senoidal de 4 puntos obtendremos los siguientes puntos del vector

$$\begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ -1 \end{bmatrix}$$

y como se ve en la siguiente imagen



Por lo que con la matriz ya simplificada y el vector de la senoidal, se obtiene lo siguiente

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 + 1 + 0 - 1 \\ 0 - 1i + 0 - 1i \\ 0 - 1 + 0 + 1 \\ 0 + i + 0 + i \end{bmatrix} = \begin{bmatrix} 0 \\ -2i \\ 0 \\ 2i \end{bmatrix}$$

De resultado se obtuvo un vector de magnitudes imaginarias. Es aquí donde nos damos cuenta que la DFT no solo mide la amplitud o magnitud, sino que también mide la fase de la señal, lo que se refleja en la parte imaginaria del resultado. Podemos representar entonces la señal transformada mediante la siguiente ecuación

$$X[k] = |X[k]|e^{j\phi_k}$$

Podemos representar cada coeficiente de la transformada en forma polar mediante la siguiente ecuación

$$X[k] = |X[k]| \cdot e^{j\phi_k}$$

donde $|X[k]|$ representa la magnitud de la componente frecuencial k , mientras que ϕ_k representa su fase.

La magnitud se calcula como

$$|X[k]| = \sqrt{(\text{Parte Real de } X[k])^2 + (\text{Parte Imaginaria de } X[k])^2}$$

y la fase mediante

$$\phi_k = \tan^{-1} \left(\frac{\text{Parte Imaginaria de } X[k]}{\text{Parte Real de } X[k]} \right).$$

Para el resultado obtenido

$$X = \begin{bmatrix} 0 \\ -2i \\ 0 \\ 2i \end{bmatrix},$$

las magnitudes y fases correspondientes

$$|X| = \begin{bmatrix} 0 \\ 2 \\ 0 \\ 2 \end{bmatrix}, \quad \Phi = \begin{bmatrix} 0 \\ -\frac{\pi}{2} \\ 0 \\ \frac{\pi}{2} \end{bmatrix}.$$

Por lo tanto, cada coeficiente puede expresarse en forma polar como

$$X[1] = 2e^{-j\frac{\pi}{2}}, \quad X[3] = 2e^{j\frac{\pi}{2}}.$$

Agrupando todos los coeficientes en forma vectorial:

$$X = \begin{bmatrix} 0 \\ 2e^{-j\frac{\pi}{2}} \\ 0 \\ 2e^{j\frac{\pi}{2}} \end{bmatrix}.$$

Es aquí donde introduciremos la notación de fasores para practicidad y claridad, representando cada componente en forma polar como un fasor, lo que nos permite

visualizar claramente la magnitud y fase de cada componente frecuencial. La representación en forma de fasores para el resultado obtenido sería: Los Fasores se definen como

$$\text{Fasor} = \text{Magnitud} \angle \text{Fase}$$

$$X = \begin{bmatrix} 0 \\ 2 \angle -\frac{\pi}{2} \\ 0 \\ 2 \angle \frac{\pi}{2} \end{bmatrix}$$

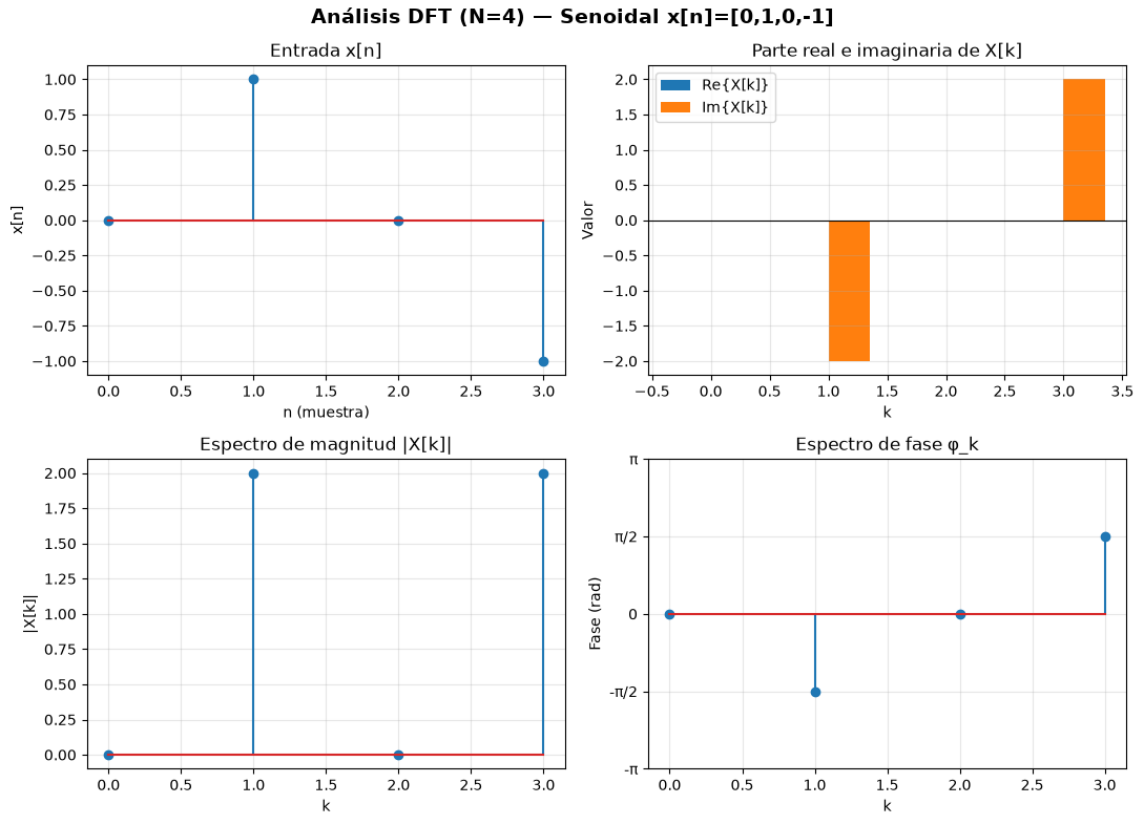


Figure 5: SinOUT

1.5.4 Pulso Cuadrado

Ya una vez con la Matriz definida ($\widehat{W}_N^{kn}$) solo quedaría sustituir el vector de la señal muestreada $\vec{x}$. Si definimos un pulso en el intervalo $n = 2$, es decir

$$\begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 3 \\ 0 \end{bmatrix}$$

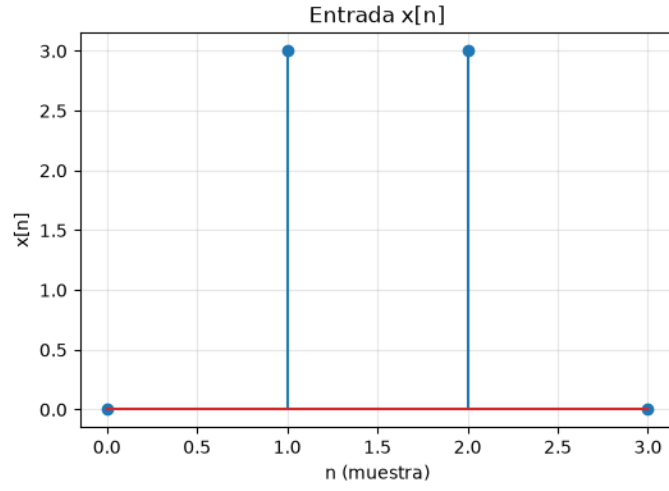


Figure 6: PulsoIN

Quedaría entonces

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix} \begin{bmatrix} 0 \\ 3 \\ 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 + 3 + 3 + 0 \\ 0 - 3i - 3 + 0 \\ 0 - 3 + 3 + 0 \\ 0 + 3i - 3 + 0 \end{bmatrix} = \begin{bmatrix} 6 \\ -(3i + 3) \\ 0 \\ 3i - 3 \end{bmatrix} = \begin{bmatrix} 6 \\ 3\sqrt{2}\angle -\frac{3\pi}{4} \\ 0 \\ 3\sqrt{2}\angle \frac{\pi}{4} \end{bmatrix}$$

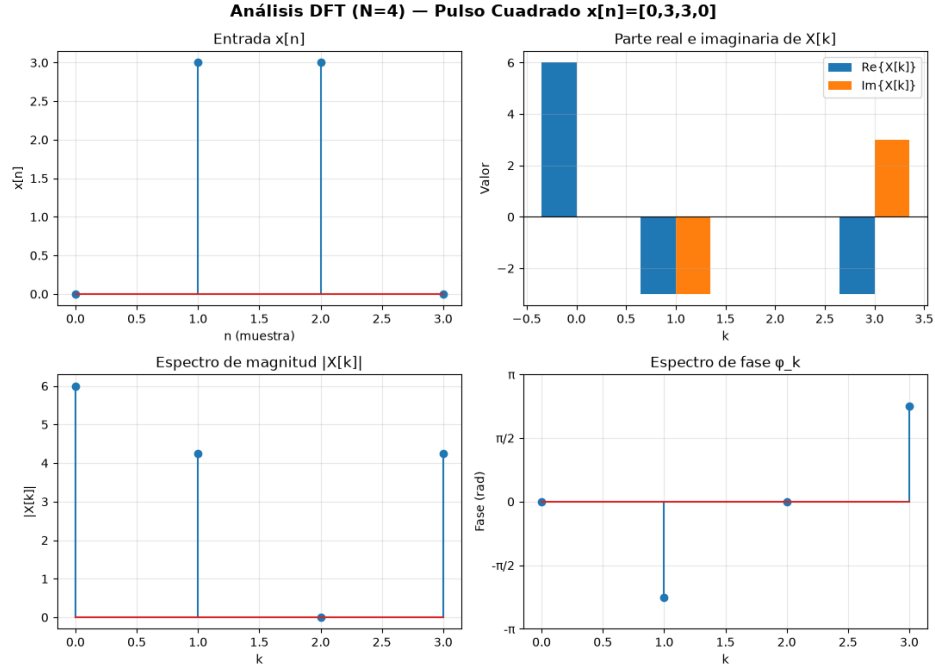


Figure 7: PulsoOUT

2 Algoritmos

2.1 ¿Qué es un algoritmo?

Un **algoritmo** se define como un conjunto ordenado, finito y bien definido de instrucciones, pasos o reglas lógicas que permiten resolver un problema determinado, realizar un cómputo, procesar datos o llevar a cabo una tarea específica. Todo algoritmo estructurado debe cumplir con tres componentes fundamentales:

- **Entrada (Input):** La información o datos iniciales requeridos para comenzar el proceso.
- **Proceso:** La secuencia de operaciones lógicas y matemáticas que transforman las entradas.
- **Salida (Output):** El resultado final u objetivo alcanzado tras la ejecución del proceso.

2.2 ¿Cuál es el algoritmo de sumar dos números?

Este constituye el ejemplo más elemental de un algoritmo cuantitativo secuencial. Dados dos valores numéricos, el procedimiento directo es el siguiente:

Algorithm 1 Suma de dos números

Require: Dos números reales, A y B .

Ensure: El resultado de la adición de ambos números.

- 1: **Inicio**
 - 2: Leer o recibir el primer número: A
 - 3: Leer o recibir el segundo número: B
 - 4: Calcular la suma de ambos valores: $Resultado \leftarrow A + B$
 - 5: Mostrar o retornar el valor de $Resultado$
 - 6: **Fin**
-

Ejemplo práctico con valores:

- **Entrada:** Supongamos que $A = 5.5$ y $B = 3.2$.
- **Proceso:**
 - Se lee $A = 5.5$.
 - Se lee $B = 3.2$.
 - Se calcula la operación: $Resultado \leftarrow 5.5 + 3.2 = 8.7$.
- **Salida:** El algoritmo muestra o retorna el valor 8.7.

2.3 ¿Cuál es el algoritmo de multiplicar dos números?

Si no tenemos el operador multiplicativo y se asume una implementación basada en operaciones fundamentales, la multiplicación se define como una sucesión de sumas acumuladas:

Algorithm 2 Multiplicación mediante sumas sucesivas

Require: Dos números enteros positivos, el multiplicando A y el multiplicador B .

Ensure: El producto $Producto = A \times B$.

```
1: Inicio
2: Leer los valores de  $A$  y  $B$ 
3: Inicializar la variable acumuladora:  $Producto \leftarrow 0$ 
4: Inicializar el contador de iteraciones:  $i \leftarrow 0$ 
5: while  $i < B$  do
6:   Acumular el valor de  $A$ :  $Producto \leftarrow Producto + A$ 
7:   Incrementar el contador:  $i \leftarrow i + 1$ 
8: end while
9: Mostrar o retornar el valor de  $Producto$ 
10: Fin
```

Ejemplo práctico con valores:

- **Entrada:** Supongamos que $A = 4$ y $B = 3$.
- **Proceso (Seguimiento del bucle):**
 - Inicialización: $Producto = 0$, $i = 0$.
 - **Iteración 1:** ¿ $0 < 3$? Sí.
 - * $Producto \leftarrow 0 + 4 = 4$
 - * $i \leftarrow 0 + 1 = 1$
 - **Iteración 2:** ¿ $1 < 3$? Sí.
 - * $Producto \leftarrow 4 + 4 = 8$
 - * $i \leftarrow 1 + 1 = 2$
 - **Iteración 3:** ¿ $2 < 3$? Sí.
 - * $Producto \leftarrow 8 + 4 = 12$
 - * $i \leftarrow 2 + 1 = 3$
 - **Evaluación final:** ¿ $3 < 3$? No. El bucle termina.
- **Salida:** Retorna el valor final de $Producto$, que es 12.

2.4 ¿Cuál es el algoritmo de sumar dos matrices?

Para realizar la adición de dos matrices, A y B , es una condición obligatoria que posean las mismas dimensiones (mismo número de filas m y columnas n). La operación se ejecuta de manera posicional elemento a elemento mediante ciclos anidados:

Algorithm 3 Suma de dos matrices

Require: Matrices A y B de dimensiones $m \times n$.

Ensure: Matriz resultante C de dimensiones $m \times n$, donde $C[i][j] = A[i][j] + B[i][j]$.

```
1: Inicio
2: Validar que las dimensiones de  $A$  y  $B$  sean idénticas ( $m \times n$ ).
3: Declarar una nueva matriz  $C$  de dimensiones  $m \times n$ .
4: for cada fila  $i$  desde 0 hasta  $m - 1$  do
5:   for cada columna  $j$  desde 0 hasta  $n - 1$  do
6:     Calcular:  $C[i][j] \leftarrow A[i][j] + B[i][j]$ 
7:   end for
8: end for
9: Retornar la matriz resultante  $C$ 
10: Fin
```

Ejemplo práctico con valores:

- **Entrada:** Sean dos matrices de dimensiones 2×2 ($m = 2, n = 2$):

$$A = \begin{pmatrix} 1 & 3 \\ 0 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 1 \\ 4 & -2 \end{pmatrix}$$

- **Proceso:** Se recorre cada posición (i, j) sumando sus elementos directos:

– Fila $i = 0$:

* Columna $j = 0$: $C[0][0] = A[0][0] + B[0][0] = 1 + 5 = 6$

* Columna $j = 1$: $C[0][1] = A[0][1] + B[0][1] = 3 + 1 = 4$

– Fila $i = 1$:

* Columna $j = 0$: $C[1][0] = A[1][0] + B[1][0] = 0 + 4 = 4$

* Columna $j = 1$: $C[1][1] = A[1][1] + B[1][1] = 2 + (-2) = 0$

- **Salida:** Se obtiene la matriz resultante:

$$C = \begin{pmatrix} 6 & 4 \\ 4 & 0 \end{pmatrix}$$

2.5 ¿Cuál es el algoritmo de multiplicar dos matrices?

Para que el producto de dos matrices $A \times B$ sea matemáticamente válido, el número de columnas de la matriz A (n) debe coincidir rigurosamente con el número de filas de la matriz B (n). Si la matriz A es de tamaño $m \times n$ y la matriz B es de tamaño $n \times p$, la matriz producto C resultante tendrá dimensiones $m \times p$.

El algoritmo implementa el cálculo del producto punto de las filas de A por las columnas de B mediante tres estructuras cíclicas anidadas:

Algorithm 4 Multiplicación de dos matrices

Require: Matriz A de dimensiones $m \times n$ y Matriz B de dimensiones $n \times p$.

Ensure: Matriz resultante C de dimensiones $m \times p$.

```
1: Inicio
2: Validar que el número de columnas de  $A$  sea igual al número de filas de  $B$ .
3: Declarar la matriz  $C$  de dimensiones  $m \times p$  e inicializar todos sus elementos en 0.
4: for cada fila  $i$  desde 0 hasta  $m - 1$  do
5:   for cada columna  $j$  desde 0 hasta  $p - 1$  do
6:     for cada elemento de acoplamiento  $k$  desde 0 hasta  $n - 1$  do
7:       Calcular el acumulado:  $C[i][j] \leftarrow C[i][j] + (A[i][k] \times B[k][j])$ 
8:     end for
9:   end for
10: end for
11: Retornar la matriz resultante  $C$ 
12: Fin
```

Ejemplo práctico con valores:

- **Entrada:** Sea A de 2×2 y B de 2×2 ($m = 2, n = 2, p = 2$):

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Matriz C se inicializa en ceros.

- **Proceso (Cálculo por posiciones usando el índice k):**
 - **Para** $C[0][0]$ ($i = 0, j = 0$):
 - * $k = 0$: $C[0][0] = 0 + (A[0][0] \times B[0][0]) = 1 \times 5 = 5$
 - * $k = 1$: $C[0][0] = 5 + (A[0][1] \times B[1][0]) = 5 + (2 \times 7) = 5 + 14 = 19$
 - **Para** $C[0][1]$ ($i = 0, j = 1$):
 - * $k = 0$: $C[0][1] = 0 + (A[0][0] \times B[0][1]) = 1 \times 6 = 6$
 - * $k = 1$: $C[0][1] = 6 + (A[0][1] \times B[1][1]) = 6 + (2 \times 8) = 6 + 16 = 22$

- **Para** $C[1][0]$ ($i = 1, j = 0$):
 - * $k = 0 : C[1][0] = 0 + (A[1][0] \times B[0][0]) = 3 \times 5 = 15$
 - * $k = 1 : C[1][0] = 15 + (A[1][1] \times B[1][0]) = 15 + (4 \times 7) = 15 + 28 = 43$
- **Para** $C[1][1]$ ($i = 1, j = 1$):
 - * $k = 0 : C[1][1] = 0 + (A[1][0] \times B[0][1]) = 3 \times 6 = 18$
 - * $k = 1 : C[1][1] = 18 + (A[1][1] \times B[1][1]) = 18 + (4 \times 8) = 18 + 32 = 50$
- **Salida:** Matriz resultante final:

$$C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

3 Definición de Complejidad Computacional

La complejidad computacional es una medida matemática que describe la cantidad de recursos que necesita un algoritmo para resolver un problema conforme aumenta el tamaño de la entrada.

Los recursos más estudiados son:

- Tiempo computacional.
- Espacio computacional.

Formalmente, la complejidad temporal se representa mediante una función:

$$T(n)$$

donde:

- n representa el tamaño de la entrada.
- $T(n)$ representa el número de operaciones requeridas.

3.1 Objetivo del Análisis de Complejidad

El análisis de complejidad no busca medir segundos reales de ejecución, ya que estos dependen del hardware y del lenguaje utilizado.

El objetivo es estudiar el **crecimiento asintótico** del algoritmo conforme aumenta el tamaño de la entrada.

Para ello se utiliza la notación Big-O:

$$T(n) = O(f(n))$$

Esta notación representa una cota superior del crecimiento del algoritmo.

3.2 Definición Formal de Big-O

Se dice que una función $T(n)$ pertenece a $O(f(n))$ si existen constantes positivas c y n_0 tales que:

$$T(n) \leq c f(n) \quad \forall n \geq n_0$$

Esto significa que, a partir de cierto tamaño de entrada, el crecimiento de $T(n)$ no supera una constante multiplicativa de $f(n)$.

3.3 Ejemplo de Complejidad Lineal

Considérese el siguiente algoritmo:

```
for i in range(n):  
    print(i)
```

El ciclo se ejecuta exactamente n veces.

La función temporal puede expresarse como:

$$T(n) = a + bn$$

donde:

- a representa operaciones constantes.
- b representa las operaciones realizadas en cada iteración.

Cuando n crece, el término lineal domina el comportamiento del algoritmo:

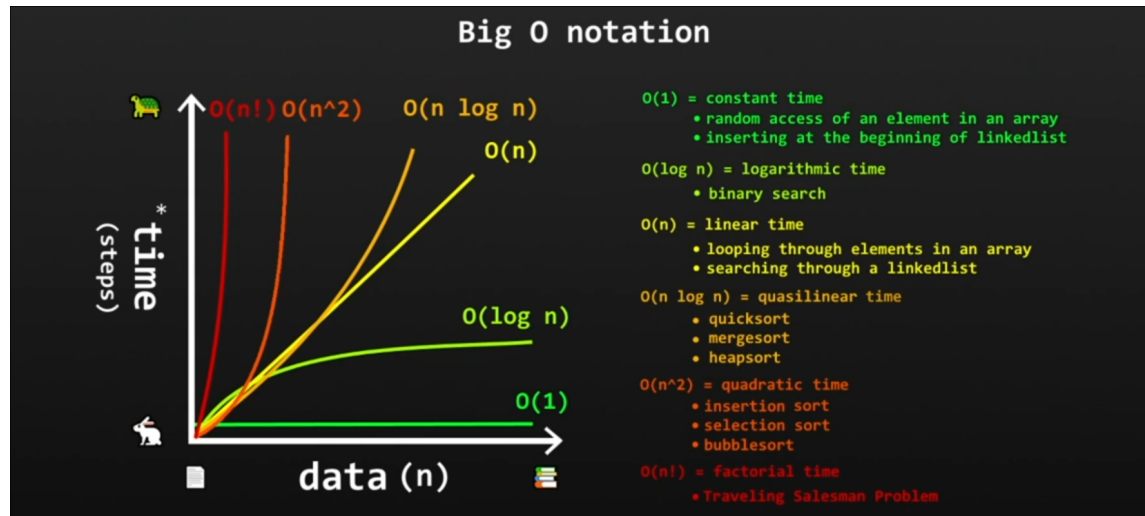
$$T(n) = O(n)$$

Las Big-O más comunes (De mejor a peor)

A continuación se presenta la tabla con las complejidades más comunes en el análisis de algoritmos:

Notación	Nombre	Qué significa en la vida real
$O(1)$	Constante	El algoritmo tarda lo mismo con 1 dato o con un billón.
$O(\log n)$	Logarítmico	Súper eficiente. El problema se divide a la mitad en cada paso.
$O(n)$	Lineal	El esfuerzo crece uno a uno con los datos.
$O(n \log n)$	Lineal-Logarítmico	El estándar de oro para algoritmos de ordenamiento eficientes (como MergeSort).
$O(n^2)$	Cuadrático	Ya empieza a doler. Si duplicas los datos, tarda 4 veces más.
$O(2^n)$	Exponencial	El terror de la computación. Un dato más y el tiempo se duplica. Inviabile para problemas grandes.

Table 1: Tabla de complejidades asintóticas Big-O.



3.4 Importancia

La complejidad computacional permite:

- Comparar algoritmos.
- Estimar escalabilidad y evaluar eficiencia.
- Optimizar recursos computacionales.

4 Complejidad de la DFT

4.1 Cantidad Real de Operaciones en la DFT

Para calcular un único valor ($X(k)$), la ecuación requiere evaluar una suma de (N) términos complejos.

Desarrollando parcialmente la expresión:

$$X(k) = x(0)e^{-j\frac{2\pi}{N}(0)k} + x(1)e^{-j\frac{2\pi}{N}(1)k} + \dots + x(N-1)e^{-j\frac{2\pi}{N}(N-1)k}$$

Cada término de la suma implica principalmente:

- una multiplicación compleja,
- y posteriormente una suma compleja acumulativa.

Por lo tanto, para calcular un único ($X(k)$):

- se realizan aproximadamente (N) multiplicaciones complejas,
- y aproximadamente ($N-1$) sumas complejas.

Así, el número total de operaciones para una sola salida puede aproximarse como:

$$N + (N - 1)$$

lo cual equivale a:

$$2N - 1$$

operaciones principales.

Sin embargo, la DFT completa no calcula solamente un valor ($X(k)$), sino todos los valores:

$$X(0), X(1), X(2), \dots, X(N-1)$$

Es decir, existen (N) salidas distintas.

Por tanto, el número total aproximado de operaciones es:

$$N(2N - 1)$$

Desarrollando:

$$2N^2 - N$$

Por ello, una expresión más precisa del costo computacional de la DFT sería:

$$T(N) = 2N^2 - N$$

No obstante, en análisis de complejidad computacional se utiliza notación asintótica, la cual estudia únicamente el comportamiento dominante cuando (N) crece mucho.

En la función:

$$T(N) = 2N^2 - N$$

el término dominante es:

$$2N^2$$

ya que para valores grandes de (N) , el término lineal $((N))$ resulta despreciable comparado con (N^2) .

Además, en notación Big-O se ignoran constantes multiplicativas, por lo que:

$$2N^2 \rightarrow N^2$$

Finalmente, la complejidad temporal de la DFT se expresa como:

$$O(N^2)$$

Es importante notar que esto no significa que el algoritmo realice exactamente (N^2) operaciones, sino que el crecimiento del número de operaciones está acotado superiormente por un término proporcional a (N^2) .

La notación Big-O simplifica la expresión eliminando:

- constantes multiplicativas,
- términos de menor orden,
- detalles que no afectan el crecimiento asintótico.

Por esta razón:

$$2N^2 - N \quad \text{y} \quad 5N^2 + 100N + 7$$

pertenecen ambos a:

$$O(N^2)$$

ya que el término cuadrático domina el crecimiento total de la función.

4.2 Interpretación

La complejidad cuadrática significa que si el tamaño de la señal aumenta, el número de operaciones crece muy rápidamente.

Por ejemplo:

- Si ($N=4$):

““

$$4^2 = 16$$

operaciones aproximadas.

- Si $N = 1000$:

$$1000^2 = 1\,000\,000$$

operaciones aproximadas. ““

Esto provoca que la DFT directa sea costosa para señales grandes.

4.3 FFT

4.3.1 Algoritmo

4.4 implementación en un microcontrolador

5 Anexos

I Transformada de Fourier Completa

Generalmente la transformada de fourier se define usando dos constantes arbitrarias a y b quedando en la siguiente forma:

$$\tilde{f}(\omega) = \sqrt{\frac{|b|}{(2\pi)^{1-a}}} \int_{-\infty}^{\infty} f(t) e^{ibwt} dt \quad y \quad f(t) = \sqrt{\frac{|b|}{(2\pi)^{1+a}}} \int_{-\infty}^{\infty} \tilde{f}(\omega) e^{-ibwx} dw \quad (7)$$

Los valores de a y b determinan la convención utilizada para la transformada de Fourier. La elección más común es $a=0$ y $b = -2\pi$, lo que da lugar a la forma estándar de la transformada de Fourier que se ha utilizado en este documento [Eq 4]. Sin embargo, otras convenciones también son válidas y pueden ser útiles en diferentes contextos. [1]

Table 2: Convenciones usadas en la transformada de Fourier

(a,b)	Uso típico
(0,1)	Física moderna
(1,-1)	Matemáticas e ingeniería de sistemas
(1,1)	Teoría de probabilidad
(-1,1)	Física clásica
(0,-2 π)	Procesamiento de señales

II Linealidad de la transformada de Fourier

Usando la definición de la transformada de Fourier vista en la ecuación ??, y con $f(x)$ y $g(x)$ dos funciones con transformadas de fourier $F(k)$ y $G(k)$, y a y b dos constantes cualesquiera, se tiene que:

$$\begin{aligned} & \int_{-\infty}^{\infty} (af(x) + bg(x)) e^{-2\pi i k x} dx = \\ &= a \int_{-\infty}^{\infty} f(x) e^{-2\pi i k x} dx + b \int_{-\infty}^{\infty} g(x) e^{-2\pi i k x} dx = \\ &= \boxed{aF(k) + bG(k)} \end{aligned}$$

III Convolución

La convolucion se define como una integral que expresa la cantidad de solapacion con otra función a medida que se desplaza. Para funciones continuas, la convolución de dos funciones f y g se define como:

$$(f * g)(t) = \int_0^{\infty} f(\tau) g(t - \tau) d\tau$$

La figura 8 muestra una representación gráfica de la convolución de dos funciones f y g . La animación de la convolución se puede visualizar como el proceso de deslizar una función sobre otra y calcular el área de superposición en cada posición. La función resultante $(f * g)(t)$ representa la cantidad de solapamiento entre las dos funciones a medida que se desplazan una sobre la otra. [7, 8]

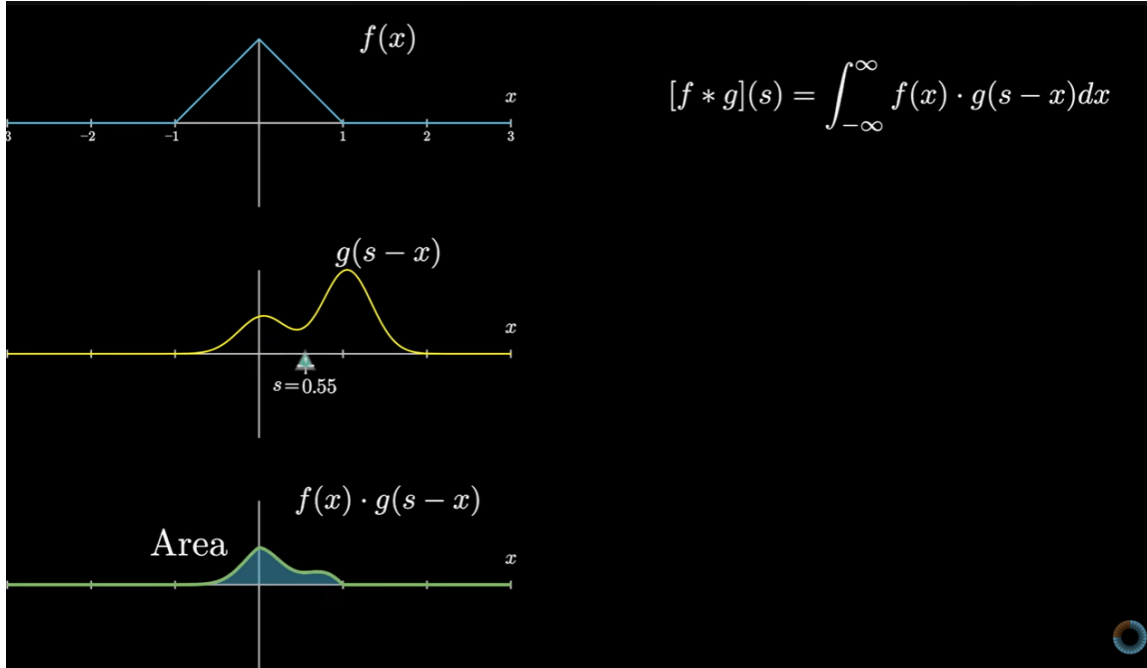


Figure 8: Convolución de dos funciones f y g

IV Construcción de la Matriz de Fourier

Para construir la matriz de Fourier, nos basaremos en un ejemplo dado por el Dr Steve L Brunton en donde analiza una señal continua de forma directa tal como se ve en la figura 9, se puede apreciar que cada punto rosa es una discretización de la señal continua, lo que da lugar a una señal discreta que se puede analizar utilizando la DFT. Iremos construyendo poco a poco la matriz de Fourier a medida que se van calculando los elementos de la DFT, empezando por la entrada y salida como lo muestra en la ecuación 8.

$$\begin{bmatrix} \hat{f}_0 \\ \hat{f}_1 \\ \hat{f}_2 \\ \vdots \\ \hat{f}_{N-1} \end{bmatrix} = [M] \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{bmatrix} \quad (8)$$

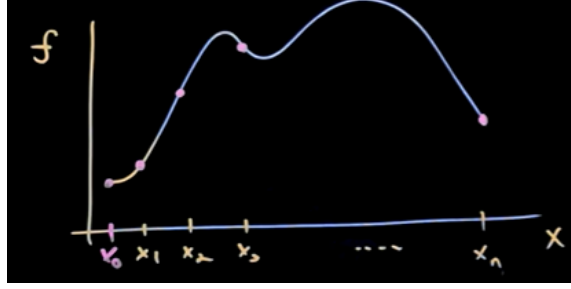


Figure 9: Señal continua a analizar

Para la Matriz M, cada elemento $M_{j,j}$ se calcula evaluando el factor de w^{jk} , lo que da lugar a la siguiente matriz de Fourier:

$$M = \begin{bmatrix} \omega^{0 \cdot 0} & \omega^{0 \cdot 1} & \omega^{0 \cdot 2} & \dots & \omega^{0 \cdot (N-1)} \\ \omega^{1 \cdot 0} & \omega^{1 \cdot 1} & \omega^{1 \cdot 2} & \dots & \omega^{1 \cdot (N-1)} \\ \omega^{2 \cdot 0} & \omega^{2 \cdot 1} & \omega^{2 \cdot 2} & \dots & \omega^{2 \cdot (N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \omega^{(N-1) \cdot 0} & \omega^{(N-1) \cdot 1} & \omega^{(N-1) \cdot 2} & \dots & \omega^{(N-1) \cdot (N-1)} \end{bmatrix}$$

Desarrollando obtenemos la siguiente matriz.

$$M = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \dots & \omega^{(N-1)(N-1)} \end{bmatrix}$$

Lo que nos permite reemplazar en la ecuacion 8 y obtener la DFT de la señal discreta.

$$\begin{bmatrix} \hat{f}_0 \\ \hat{f}_1 \\ \hat{f}_2 \\ \vdots \\ \hat{f}_{N-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \dots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \dots & \omega^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{N-1} \end{bmatrix}$$

References

- [1] Bracewell, R. The Fourier Transform and Its Applications, 3rd ed. New York: McGraw-Hill, 1999.
- [2] ScienceDirect (2024) *Integral Transform* Recuperado de <https://www.sciencedirect.com/topics/mathematics/integral-transform>

- [3] Britannica (2024) *Integral Transform* Recuperado de <https://www.britannica.com/science/integral-transform>
- [4] StudySmarter (2024) *Transformadas Integrales* Recuperado de <https://www.studysmarter.es/resumenes/ingenieria/ingenieria-electrica/transformadas-integrales/>
- [5] GeeksforGeeks (2024) *Fourier Transform* Recuperado de <https://www.geeksforgeeks.org/maths/fourier-transform/>
- [6] Wolfram MathWorld (2024) *Discrete Fourier Transform* Recuperado de <https://mathworld.wolfram.com/DiscreteFourierTransform.html>
- [7] Wolfram MathWorld (2024) *Convolution* Recuperado de <https://mathworld.wolfram.com/Convolution.html>
- [8] 3Blue1Brown (2024) *But what is a convolution?* Recuperado de <https://www.youtube.com/watch?v=IaSGqQa50-M>
- [9] Taboga, Marco (2021). "Discrete Fourier transform", Lectures on matrix algebra. <https://www.statlect.com/matrix-algebra/discrete-Fourier-transform>.